آزمایش دوم: پیاده سازی پردازنده ARM – جلسه اول

گرد آورنده: امید قلی‌زاده – امیررضا غلامی

قسمت اول آزمایش ARM: (جلسه اول)

در ابتدا باید ۵ مرحله خط لوله پردازنده را به همراه مرحله واکشی به صورت کاملاً ساده پیاده سازی کنید، برای این کار به روش زیر عمل نمایید:

۱- در ابتدا یک فایل به نام ARM.v ایجاد کرده و ماژولی به همین نام بنویسید. این ماژول تا اخر آزمایشگاه به عنوان ماژول Top-level شما برای شبیه سازی استفاده می‌شود.

۲- برای هر کدام از ۵ مرحله خط لوله یک ماژول مجزا ایجاد کنید.

۳- ماژول واکشی و رجیستر بعد از آن را با ورودی های زیر ایجاد کنید.

```
1
2  module IF_Stage (
3      input clk,rst,freeze,Branch_taken ,
4      input[31:0] BranchAddr,
5      output [31:0] PC,Instruction
6  );
7
```

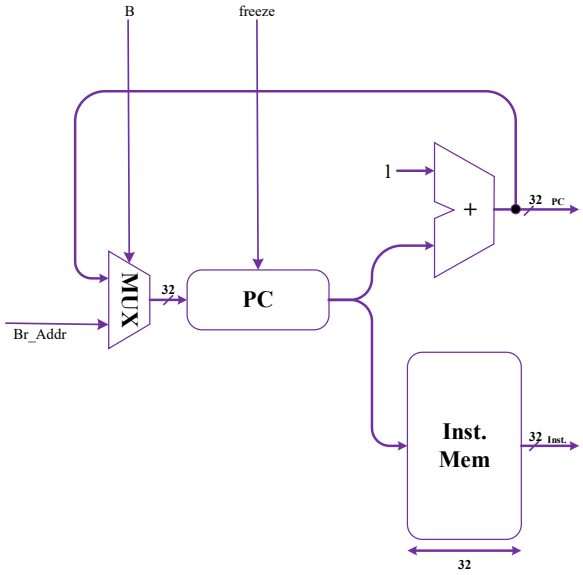
```
1  module IF_Stage_Reg (
2      input clk,rst,freeze,flush,
3      input[31:0]PC_in, Instruction_in,
4      output reg [31:0] PC,Instruction
5  );
6
```

۴- ماژول کدگشایی(ID Stage)، اجرا (EXE Stage)، حافظه (MEM Stage) و بازنشانی (WB Stage) به همراه رجیسترهای بعد از آن را تنها با ورودی‌های زیر ایجاد کنید توجه داشته باشید که خروجی رجیسترهای هر مرحله باید از نوع output reg باشد:

```
(
   input clk,
   input rst,
   input [31:0] PC_in,
   output[31:0] PC,
);
```

ماژول واکشی دستورالعمل دارای یک رجیستر PC به همراه یک حافظه داده‌ای به عنوان حافظه دستورالعمل است.

۵- در ادامه می‌خواهیم واحد واکشی دستور (IF Stage) را طراحی و تست کنیم. شماتیک کلی این واحد به صورت شکل زیر است. وظیفه این واحد، خواندن (واکشی) دستورالعمل‌ها از حافظه، یکی پس از دیگری، برای ارسال به مرحله بعد است.



اجزای کلیدی این مرحله به صورت زیر است:

❖ شمارنده برنامه(Program Counter – PC) :

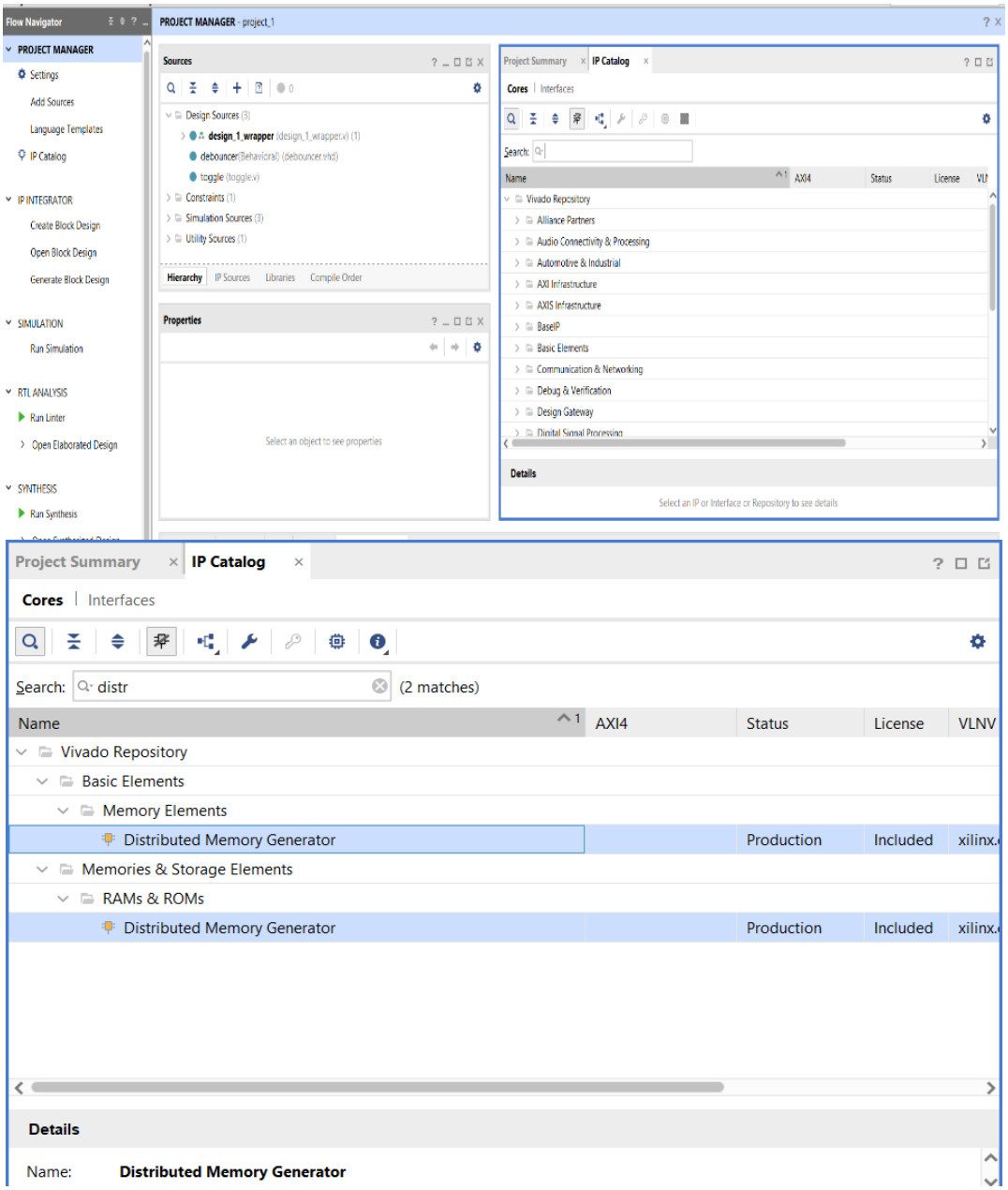
- رجیستر PC مانند یک شمارنده عمل می کند، که از صفر شروع به شمارش می کند، تا به بیشینه مقدار خود برسد (رجیستر PC را ۳۲ بیتی در نظر بگیرید). اگر کلید Reset زده شود، مقدار PC صفر می‌شود. همچنین در صورت ۱ بودن freeze مقدار ورودی در رجیستر بارگذاری نمی شود.
- باید با استفاده از HDL، آن را طراحی و پیاده‌سازی کنید.

❖ حافظه دستور العمل(Instruction Memory) :

- آدرس را از PC دریافت می‌کند و دستورالعمل ذخیره شده در آن آدرس را خروجی می‌دهد.
- برای پیاده سازی این قسمت، لازم است با استفاده از گزینه IP Catalog، گزینه Distributed Memory Generator را جستجو کرده و تنظیمات آن را به صورت شکل زیر انجام دهید.

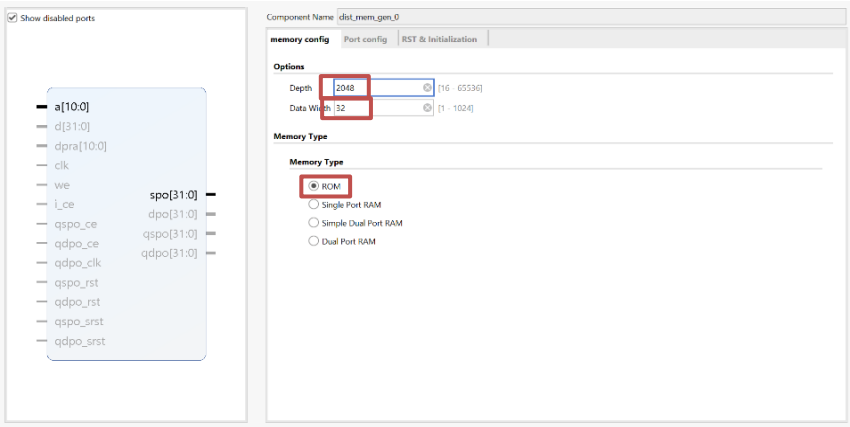
آزمایش دوم: پیاده سازی پردازنده ARM – جلسه اول

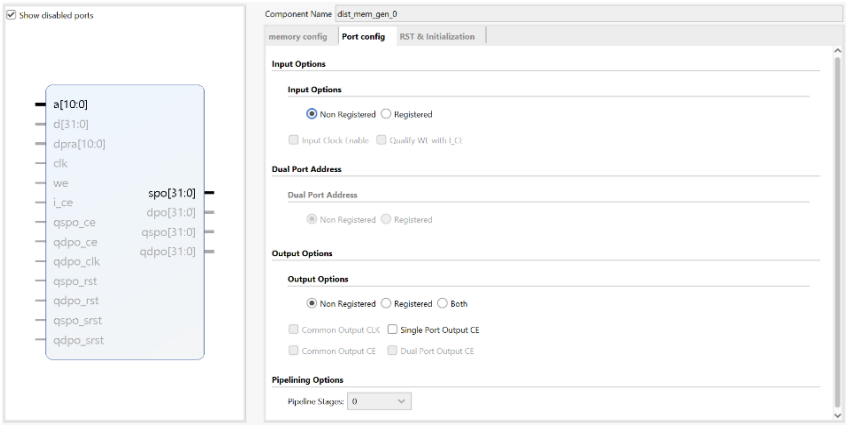
گرد آورنده: امید قلی‌زاده – امیررضا غلامی

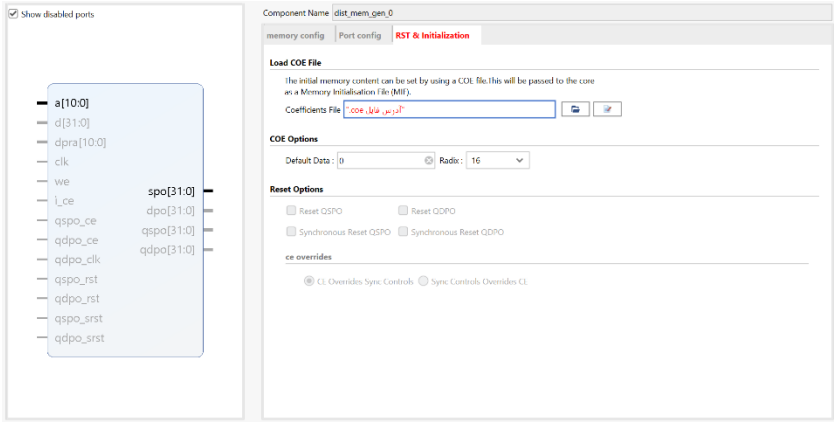


آزمایش دوم: پیاده سازی پردازنده ARM – جلسه اول

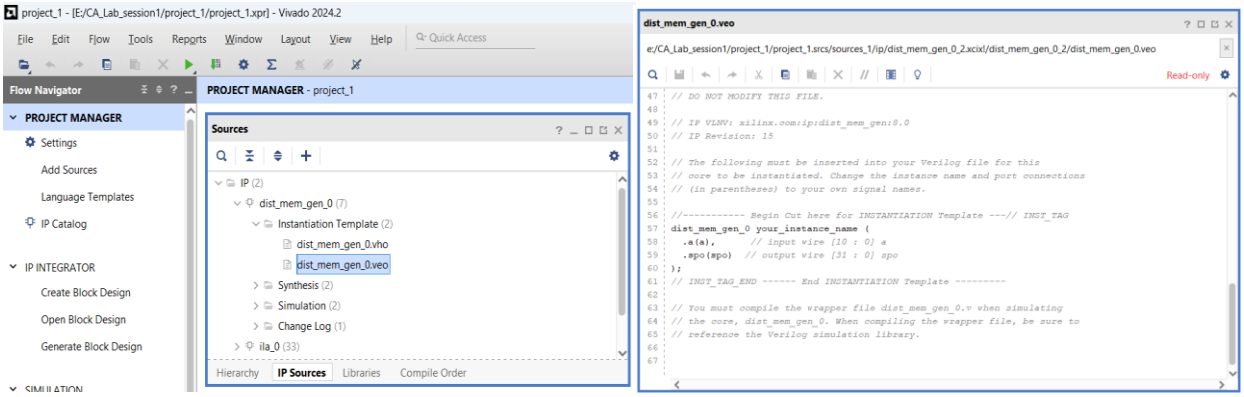
گرد آورنده: امید قلی‌زاده – امیررضا غلامی







- پس از تنظیم پارامتر ها، با رفتن به بخش IP Sources، باز کردن پوشه های نمونه IP ایجاد شده، در پوشه Instantiation Template، با باز کردن فایل veo.، می‌توانید با استفاده از نمونه کد ایجاد شده برای نمونه گیری از IPمورد نظر، طراحی خود را انجام دهید.



- فایل coe. مورد نظر بر روی سامانه قرار داده شده تا IP مورد نظر را مقداردهی اولیه نمایید. محتوای این فایل، الگوریتم مرتب سازی حبابی مذکور می‌باشد.

تذکر: با تنظیم کردن پارامتر Depth، عرض بیت مورد استفاده برای آدرس این IP را تعیین می‌کنید. بنابراین توجه کنید که برای انجام اتصالات، نیاز است Sliceای از بیت‌های خروجی PC را به IM به عنوان آدرس دهید.

❖ واحد فزاینده (Adder)

- وظیفه این واحد، محاسبه آدرس دستورالعمل بعدی است.
- در این آزمایشگاه، از آنجایی که حافظه ها Word Addressable هستند، این واحد، آدرس فعلی PC را با ۱ جمع می‌کند (PC + 1).
- باید با استفاده از HDL، آن را طراحی و پیاده‌سازی کنید.

۶- از ماژول IF در Top-Level نمونه گیری کنید.

۷- ماژول واکشی دارای پورت ورودی کلاک و ریست است. همچنین دو پورت خروجی دارد، که یکی خروجی دستورالعمل است که به مرحله دیکد(ID Stage) می رود. مقدار PC را برای مشاهده همراه با دستور به مراحل بعد ارسال نمایید.

۸- مقادیر ورودی freeze، Branch_tacken، BranchAddr را به صفر assign کنید.

۹- چهار ماژول باقیمانده به همراه رجیسترهای میانی هیچ عملیاتی انجام نمی‌دهند، و تنها مقادیر ورودی را به خروجی ارسال می‌کنند تا حرکت موج‌گونه دستورات در رجیسترهای میانی مراحل مشاهده شود.

۱۰- در ماژول Top-Level، از ماژول‌های بالا نمونه‌گیری کنید. ماژول‌ها را به یکدیگر متصل نمایید.

پیاده سازی روی بورد:

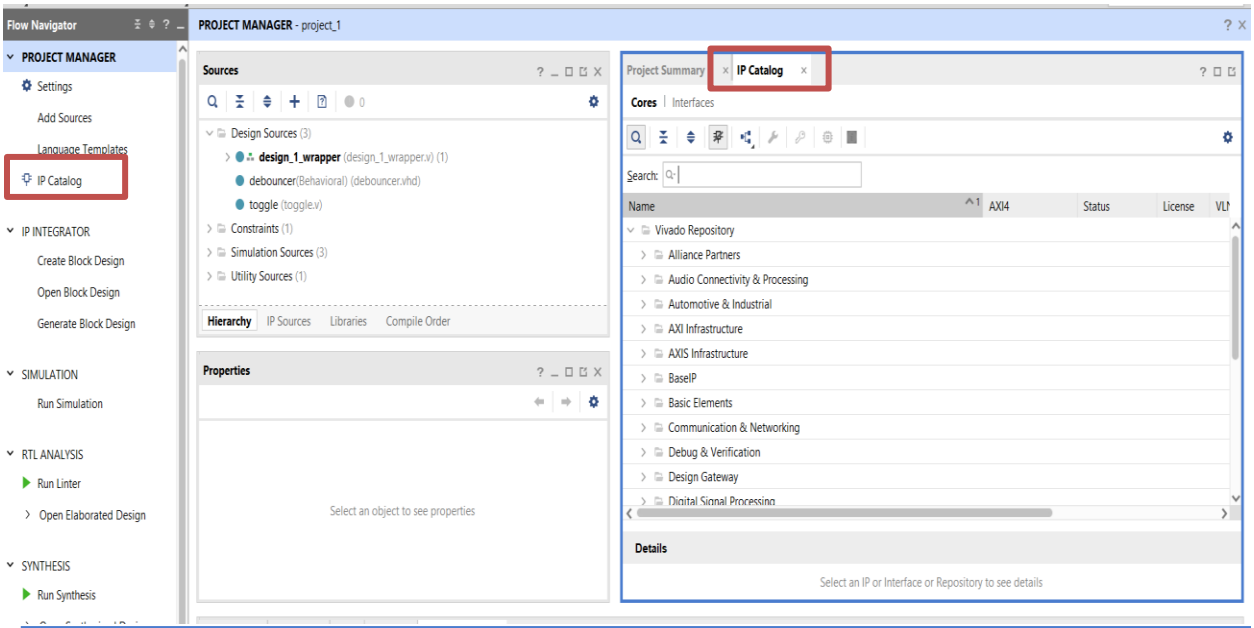
۱. ایجاد و پیکربندی Integerated Logic Analyzer (ILA)

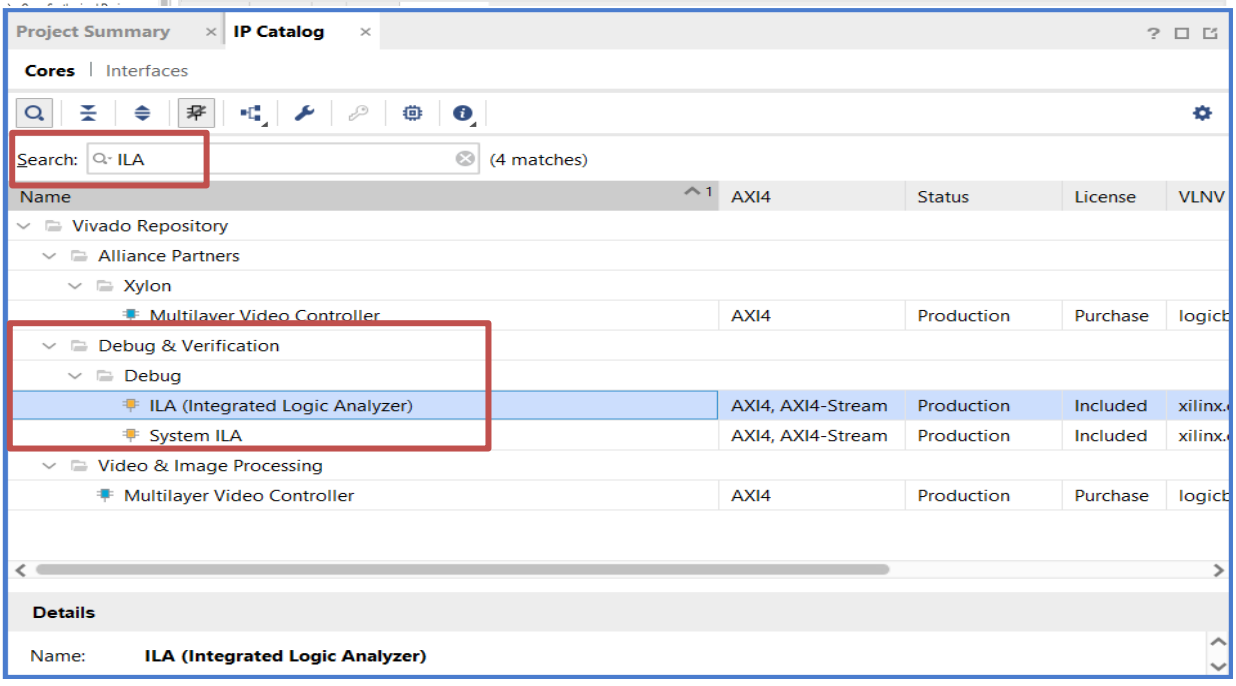
ILA یکی از ابزارهای بسیار کاربردی در طراحی‌های FPGA است که برای مشاهده و تحلیل سیگنال‌های داخلی مدار به‌صورت درون‌تراشه‌ای (on-chip debugging) استفاده می‌شود. برخلاف اسیلوسکوپ یا منطق‌سنج‌های خارجی، ILA مستقیماً در طراحی FPGA قرار می‌گیرد و می‌تواند سیگنال‌های داخلی را بدون نیاز به پین‌های خارجی نمونه‌برداری و نمایش دهد. با استفاده از نرم‌افزار Vivado، کاربر می‌تواند نقاط مورد نظر (probes) را به سیگنال‌های طراحی متصل کرده و پس از برنامه‌ریزی FPGA، رفتار آن‌ها را در زمان واقعی مشاهده و تحلیل کند. این ابزار برای اشکال‌زدایی (debugging)، بررسی وضعیت FSMها، تایید صحت داده‌ها در مسیرها و اطمینان از عملکرد درست ماژول‌ها بسیار مفید است.

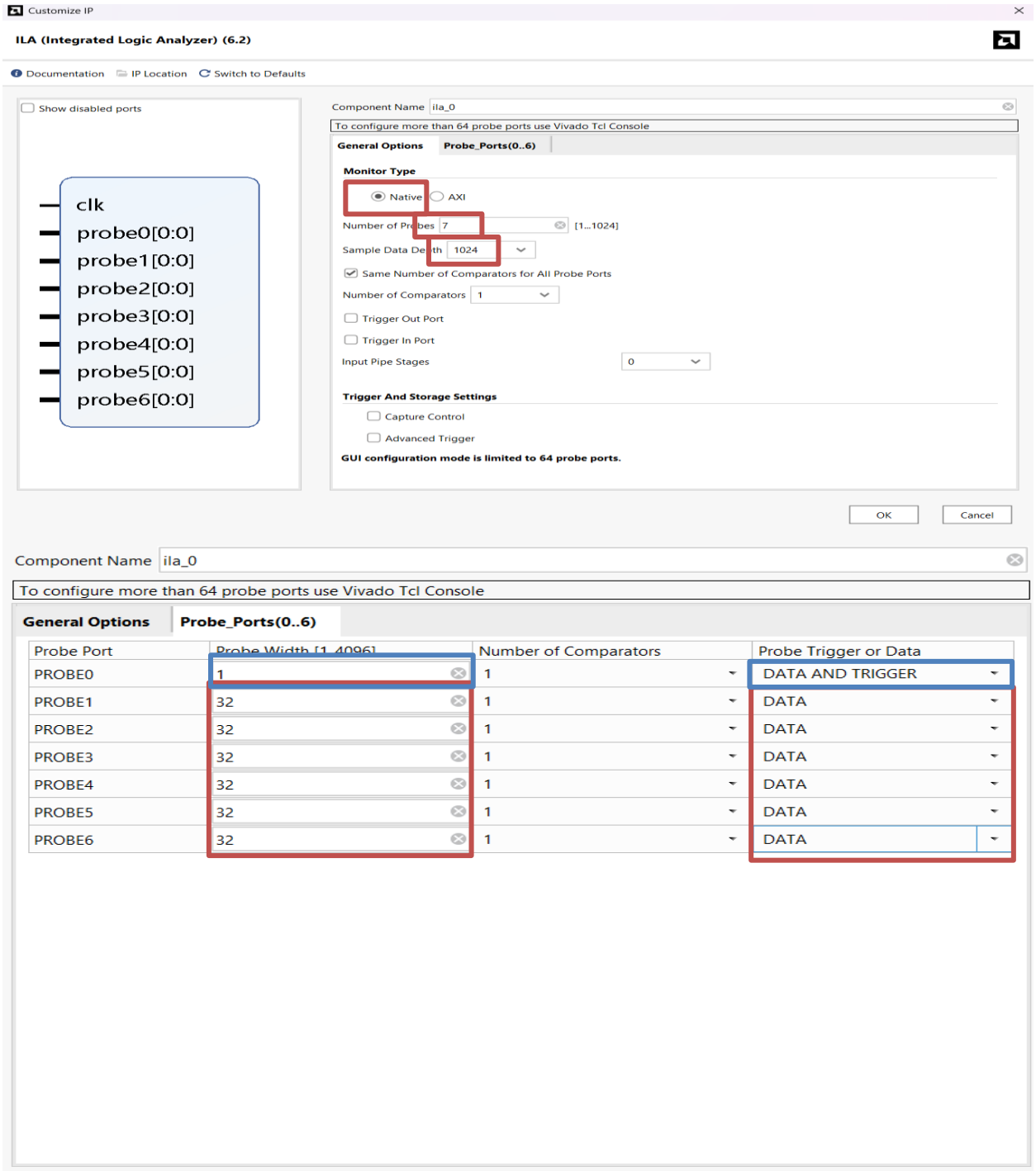
- برای اضافه کردن دیباگر سخت‌افزاری (ILA)، از منوی IP Catalog در Vivado، هسته‌ی ILA (Integrated Logic Analyzer) را جستجو و به پروژه اضافه کنید (دقیقاً مانند زمانی که IP هسته‌ی حافظه را اضافه می‌کردید).
- در پنجره تنظیمات ILA:
  - تعداد پروب‌ها (Number of Probe Ports): این عدد را برابر یکی بیشتر از تعداد سیگنال‌هایی که می‌خواهید در لحظه مشاهده کنید، تنظیم نمایید. (برای مثال، اگر می‌خواهید فقط PC و Instruction را ببینید، به ۳ پروب نیاز دارید زیرا پروب سوم سیگنال ریست در نقش Trigger خواهد بود).
  - نوع مانیتور (Monitor Type): این گزینه را روی NATIVE تنظیم کنید.
  - تنظیمات پروب (Probe Ports Tab): در این تب، عرض بیت (PROBE_WIDTH) هر پروب را دقیقاً مطابق با سیگنالی که قرار است به آن متصل شود، تنظیم کنید (مثلاً ۳۲ بیت برای PC_out یا Instruction_out).
- تنظیم تریگر (Trigger):
  - ILA نیاز به یک سیگنال تریگر دارد تا بداند از چه زمانی ضبط داده‌ها را شروع کند.
  - یک راه ساده این است که یکی از پروب‌های داده خود را به صورت DATA_AND_TRIGGER تعریف کنید و سیگنال reset کلی مدار خود را به آن متصل کنید.
- پس از اضافه کردن IP، آن را در ماژول Top_Level خود نمونه‌گیری (instantiate) کنید و سیگنال‌هایی که می‌خواهید ببینید (مانند خروجی PC از رجیسترهای میانی) را به پورت‌های probe آن متصل نمایید.

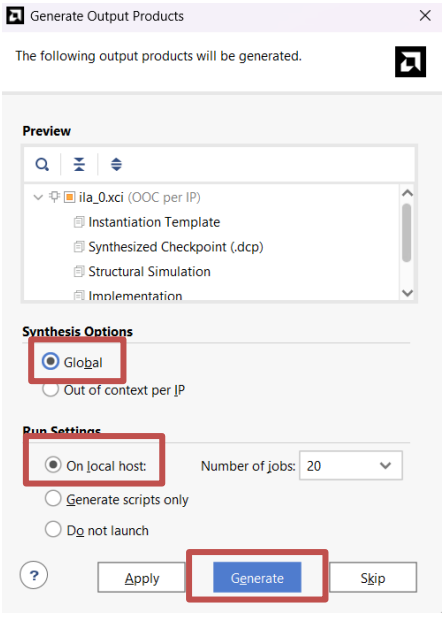
آزمایش دوم: پیاده سازی پردازنده ARM – جلسه اول

گرد آورنده: امید قلی‌زاده – امیررضا غلامی






Customize IP
ILA (Integrated Logic Analyzer) (6.2)
Documentation
IP Location
Switch to Defaults
Show disabled ports
clk
probe0[0:0]
probe1[0:0]
probe2[0:0]
probe3[0:0]
probe4[0:0]
probe5[0:0]
probe6[0:0]
Component Name ila_0
To configure more than 64 probe ports use Vivado Tcl Console
General Options
Probe_Ports(0..6)
Monitor Type
Native
AXI
Number of Probes 7
[1...1024]
Sample Data Depth 1024
Same Number of Comparators for All Probe Ports
Number of Comparators 1
Trigger Out Port
Trigger In Port
Input Pipe Stages 0
Trigger And Storage Settings
Capture Control
Advanced Trigger
GUI configuration mode is limited to 64 probe ports.
OK
Cancel
Component Name ila_0
To configure more than 64 probe ports use Vivado Tcl Console
General Options
Probe_Ports(0..6)
Probe Port
Probe Width [1..4096]
Number of Comparators
Probe Trigger or Data
PROBE0 1 1 DATA AND TRIGGER
PROBE1 32 1 DATA
PROBE2 32 1 DATA
PROBE3 32 1 DATA
PROBE4 32 1 DATA
PROBE5 32 1 DATA
PROBE6 32 1 DATA



پس از تنظیم پارامتر ها (مانند قبل)، با رفتن به بخش IP Sources، باز کردن پوشه های نمونه IP ایجاد شده، در پوشه Instantiation Template، با باز کردن فایل veo.، می‌توانید با استفاده از نمونه کد ایجاد شده برای نمونه گیری از IPمورد نظر، طراحی خود را انجام دهید.

نکته مهم: مدیریت لرزش کلید (Button Debouncing)

کلیدهای فشاری (Button) تعبیه شده بر روی بردهای FPGA (مانند Zynq) ذاتاً دارای لرزش مکانیکی و الکتریکی (Bounce) هستند. این پدیده باعث می‌شود که با یک بار فشردن کلید، چندین پالس ناخواسته (به جای یک پالس تمیز) به مدار اعمال شود. برای سیگنال حساسی مانند Reset، این لرزش مخرب است و می‌تواند مدار شما را مکرراً ریست کند.

دستورالعمل:

- برای جلوگیری از این مشکل، لازم است فقط هنگام پیاده‌سازی نهایی روی برد (و نه در شبیه‌سازی)، از ماژول Debouncer استفاده کنید.
- یک نمونه (instance) از ماژول Debouncer (که در سامانه آموزشی در اختیارتان قرار داده شده) را در فایل Top خود نمونه‌گیری (instantiate) کنید.
- پین ورودی فیزیکی که از دکمه key1 برد می‌آید را به ورودی ماژول Debouncer متصل نمایید.
- خروجی ماژول Debouncer را به عنوان سیگنال reset اصلی و سراسری برای تمام ماژول‌های دیگر مدارتان (مانند PC، رجیسترهای میانی و...) استفاده کنید.

۲. برای پیاده سازی روی بورد نیاز به ساختن یک wrapper به صورت زیر هستید که این wrapper روی بورد پیاده سازی می‌شود. ساختار این wrapper به شرح زیر است: (دقت کنید که در Testbench ای که شبیه‌سازی می‌کنید نباید از این wrapper را استفاده کنید و همان ماژول ARM را شبیه‌سازی کنید.)

```
module ARM_FPGA_wrapper(
        input clk,
        input key1
    );

    wire deb_out, rst;
    debouncer deb_inst (
        .SIGNAL_I(key1),
        .CLK_I(clk),
        .SIGNAL_O(deb_out)
    );
    assign rst = ~deb_out;

    ARM ARM_inst (
        .clk(clk),
        .rst(rst)
    );


    ila_0 ila_inst (
        .clk(clk), // input wire clk


        .probe0(rst), // input wire [0:0]  probe0
        .probe1(ARM_inst.IF_pc), // input wire [31:0]  probe1
        .probe2(ARM_inst.ID_pc), // input wire [31:0]  probe2
        .probe3(ARM_inst.EXE_pc), // input wire [31:0]  probe3
        .probe4(ARM_inst.MEM_pc), // input wire [31:0]  probe4
        .probe5(ARM_inst.WB_pc), // input wire [31:0]  probe5
        .probe6(ARM_inst.ID_instruction) // input wire [31:0]  probe6
    );


endmodule
```

۳. سنتز (Synthesis) و پیاده‌سازی (Implementation) طرح برای بورد Zynq.

۴. نمایش زنده‌ی نتایج ILA به دستیار آموزشی که "حرکت موج‌گونه" (Rippling) مقادیر PC و Instruction را در خروجی تمامی رجیسترهای میانی (IF/ID, ID/EXE, EXE/MEM, MEM/WB) نشان دهد.

نتایج و موارد تحویلی این جلسه:

۱. پیاده‌سازی IF Stage و رجیسترهای Pipe:
- لازم است که IF Stage و رجیسترهای Pipe را با توجه به توضیحات فوق، پیاده سازی و شبیه سازی کنید و نتایج را به دستیار آموزشی مربوطه نمایش دهید.

۲. شبیه‌سازی (Simulation):
- ارائه یک Testbench مناسب که سیگنال‌های clk و reset را تولید می‌کند.
- نمایش نتایج شبیه‌سازی در Vivado Simulator که نشان دهد:
- PC به درستی شمارش می‌کند (۰, ۱, ۲, ...).
- PC در صورت فعال شدن reset صفر می‌شود.
- دستورالعمل‌ها به درستی از حافظه خوانده شده و در هر پالس ساعت، یک مرحله در خط لوله به جلو حرکت می‌کنند.

۳. تأیید سخت‌افزاری روی بورد (ILA):
- سنتز (Synthesis) و پیاده‌سازی (Implementation) موفق طرح برای بورد Zynq.
- نمایش زنده‌ی نتایج ILA به دستیار آموزشی که "حرکت موج‌گونه" (Rippling) مقادیر PC و Instruction را در خروجی تمامی رجیسترهای میانی (IF/ID, ID/EXE, EXE/MEM, MEM/WB) نشان دهد.